GETTING STARTED

Quickstart

Welcome to Claude Code!

This quickstart guide will have you using AI-powered coding assistance in a few minutes. By the end, you'll understand how to use Claude Code for common development tasks.

Before you begin

Make sure you have:

- A terminal or command prompt open
  - If you've never used the terminal before, check out the **terminal guide**
- A code project to work with
- A **Claude subscription** (Pro, Max, Team, or Enterprise), **Claude Console** account, or access through a **supported cloud provider**

This guide covers the terminal CLI. Claude Code is also available on the **web**, as a **desktop app**, in **VS Code** and **JetBrains IDEs**, in **Slack**, and in CI/CD with **GitHub Actions** and **GitLab**. See **all interfaces**.

Step 1: Install Claude Code

To install Claude Code, use one of the following methods:

Native Install (Recommended)

macOS, Linux, WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Windows CMD:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd &&
install.cmd && del install.cmd
```

If you see `The token '&&' is not a valid statement separator`, you're in PowerShell, not CMD. If you see `'irm' is not recognized as an internal or external command`, you're in CMD, not PowerShell. Your prompt shows `PS C:\` when you're in PowerShell and `C:\` without the `PS` when you're in CMD.If the install command fails with `syntax error near unexpected token '<'`, a `403`, or another curl error, see **Troubleshoot installation** to match the error to a fix and for alternative install methods.**Git for Windows** is recommended on native Windows so Claude Code can use the Bash tool. If Git for Windows is not installed, Claude Code uses PowerShell as the shell tool instead. WSL setups do not need Git for Windows.

Native installations automatically update in the background to keep you on the latest version.

Homebrew

```
brew install --cask claude-code
```

Homebrew offers two casks. `claude-code` tracks the stable release channel, which is typically about a week behind and skips releases with major regressions. `claude-code@latest` tracks the latest channel and receives new versions as soon as they ship.

Homebrew installations do not auto-update. Run `brew upgrade claude-code` or `brew upgrade claude-code@latest`, depending on which cask you installed, to get the latest features and security fixes.

WinGet

```
winget install Anthropic.ClaudeCode
```

WinGet installations do not auto-update. Run `winget upgrade Anthropic.ClaudeCode` periodically to get the latest features and security fixes.

You can also install with **apt, dnf, or apk** on Debian, Fedora, RHEL, and Alpine.

Step 2: Log in to your account

Claude Code requires an account to use. Start an interactive session with the `claude` command and you'll be prompted to log in on first use:

```
claude
```

For Claude subscription or Console accounts, follow the prompts to complete authentication in your browser. To switch accounts later or re-authenticate, type `/login` inside the running session:

```
/login
```

You can log in using any of these account types:

- **Claude Pro, Max, Team, or Enterprise** (recommended)
- **Claude Console** (API access with pre-paid credits). On first login, a "Claude Code" workspace is automatically created in the Console for centralized cost tracking.
- **Amazon Bedrock, Google Vertex AI, or Microsoft Foundry** (enterprise cloud providers)

Once logged in, your credentials are stored and you won't need to log in again.

Step 3: Start your first session

Open your terminal in any project directory and start Claude Code:

```
cd /path/to/your/project
claude
```

You'll see the Claude Code prompt with the version, current model, and working directory shown above it. Type `/help` for available commands or `/resume` to continue a previous conversation.

> After logging in (Step 2), your credentials are stored on your system. Learn more in **Credential Management**.

Step 4: Ask your first question

Let's start with understanding your codebase. Try one of these commands:

```
what does this project do?
```

Claude will analyze your files and provide a summary. You can also ask more specific questions:

```
what technologies does this project use?
```

```
where is the main entry point?
```

```
explain the folder structure
```

You can also ask Claude about its own capabilities:

```
what can Claude Code do?
```

```
how do I create custom skills in Claude Code?
```

```
can Claude Code work with Docker?
```

> Claude Code reads your project files as needed. You don't have to manually add context.

Step 5: Make your first code change

Now let's make Claude Code do some actual coding. Try a simple task:

```
add a hello world function to the main file
```

Claude Code will:

1. Find the appropriate file
2. Show you the proposed changes
3. Ask for your approval
4. Make the edit

> Claude Code always asks for permission before modifying files. You can approve individual changes or enable "Accept all" mode for a session.

Step 6: Use Git with Claude Code

Claude Code makes Git operations conversational:

```
what files have I changed?
```

```
commit my changes with a descriptive message
```

You can also prompt for more complex Git operations:

```
create a new branch called feature/quickstart
```

```
show me the last 5 commits
```

```
help me resolve merge conflicts
```

Step 7: Fix a bug or add a feature

Claude is proficient at debugging and feature implementation.

Describe what you want in natural language:

```
add input validation to the user registration form
```

Or fix existing issues:

```
there's a bug where users can submit empty forms - fix it
```

Claude Code will:

- Locate the relevant code
- Understand the context
- Implement a solution
- Run tests if available

Step 8: Test out other common workflows

There are a number of ways to work with Claude:

Refactor code

```
refactor the authentication module to use async/await instead of
callbacks
```

Write tests

```
write unit tests for the calculator functions
```

Update documentation

```
update the README with installation instructions
```

Code review

```
review my changes and suggest improvements
```

Talk to Claude like you would a helpful colleague. Describe what you want to achieve, and it will help you get there.

Essential commands

Here are the most important commands for daily use. Shell commands run from your terminal to start or resume Claude Code. Session commands run inside Claude Code after it starts.

Shell commands

Command	What it does	Example
`claude`	Start interactive mode	`claude`
`claude "task"`	Run a one-time task	`claude "fix the build error"`
`claude -p "query"`	Run one-off query, then exit	`claude -p "explain this function"`
`claude -c`	Continue most recent conversation in current directory	`claude -c`
`claude -r`	Resume a previous conversation	`claude -r`

Session commands

Command	What it does	Example
`/clear`	Clear conversation history	`/clear`

Command	What it does	Example
`/help`	Show available commands	`/help`
`/exit` or Ctrl+D	Exit Claude Code	`/exit`

See the **CLI reference** for the complete list of shell commands and the **commands reference** for the complete list of session commands.

Pro tips for beginners

For more, see **best practices** and **common workflows**.

- Be specific with your requests
- Use step-by-step instructions
- Let Claude explore first
- Save time with shortcuts

What's next?

Now that you've learned the basics, explore more advanced features:

How Claude Code works

Understand the agentic loop, built-in tools, and how Claude Code interacts with your project

Best practices

Get better results with effective prompting and project setup

Common workflows

Step-by-step guides for common tasks

Extend Claude Code

Customize with CLAUDE.md, skills, hooks, MCP, and more

Getting help

- **In Claude Code**: Type `/help` or ask “how do I…”
- **Documentation**: You’re here! Browse other guides
- **Community**: Join our **Discord** for tips and support